AgriSmart — Nhận diện & tư vấn bệnh lúa theo thời gian thực (realtime)

Hệ thống nhận diện bệnh lúa qua ảnh camera bằng model học sâu (deep learning — một dạng trí tuệ nhân tạo học từ rất nhiều ảnh ví dụ để tự nhận ra quy luật, ở đây là kiến trúc **YOLO11s-cls**: mạng nơ-ron YOLO phiên bản 11, cỡ nhỏ ("s" = small), dùng cho tác vụ phân loại ảnh — classification, viết tắt "cls"). Model chạy trực tiếp trên Raspberry Pi 4 (máy tính nhúng cỡ nhỏ, giá rẻ), kèm một trang dashboard (bảng điều khiển web) hiển thị video, kết quả chẩn đoán, dữ liệu cảm biến đất/khí hậu và một chatbot tư vấn xử lý bệnh.

Dữ liệu huấn luyện: bộ **Paddy Doctor** (10.407 ảnh lá lúa, 10 lớp bệnh, thu thập tại Tamil Nadu — Ấn Độ, công bố tại hội nghị khoa học **NeurIPS 2022** — Neural Information Processing Systems, một hội nghị lớn về trí tuệ nhân tạo). Xem phân tích đầy đủ tại paddy_analysis_output/paddy_doctor_report.txt.

---

1. Thành phần & chức năng

1.1 Cây thư mục

```
benholua/
├── paddy_images/              Dataset (tập dữ liệu ảnh) gốc — 10.407 ảnh, chia
│                               theo 10 thư mục lớp bệnh (chỉ dùng để huấn luyện,
│                               KHÔNG mang theo khi triển khai thực tế)
│
├── paddy_analysis_output/     Báo cáo + biểu đồ phân tích dataset, thường gọi
│                               tắt là EDA (Exploratory Data Analysis — phân tích
│                               khám phá dữ liệu trước khi đưa vào huấn luyện)
│   ├── paddy_doctor_report.txt    Báo cáo phân tích đầy đủ (phân phối lớp, giống
│   │                               lúa, tuổi cây...)
│   ├── paddy_metadata.csv         Bảng thông tin 10.407 dòng, mỗi dòng 1 ảnh:
│   │                               nhãn bệnh, giống lúa, tuổi cây, kích thước ảnh
│   ├── *.png / *.jpg              10 biểu đồ minh hoạ (phân phối, bản đồ nhiệt
│   │                               so sánh chéo, mất cân bằng lớp...)
│   └── so_do_don_gian.md / so_do_mo_hinh.md   Sơ đồ trực quan (định dạng mermaid,
│                                   xem được trực tiếp trên GitHub hoặc VS Code)
│                                   giải thích bài toán & quy trình xử lý
│
├── result_training/           Kết quả huấn luyện model (copy về từ máy chủ có
│                               card đồ hoạ dùng để train — xem phần 2.3)
│   ├── args.yaml                   Toàn bộ tham số huấn luyện đã dùng
│   ├── results.csv                 Nhật ký độ lỗi/độ chính xác qua từng vòng
│   │                               huấn luyện (epoch — một lượt máy nhìn hết
│   │                               toàn bộ ảnh huấn luyện một lần)
│   └── confusion_matrix*.png       Ma trận nhầm lẫn — bảng đối chiếu bệnh máy
```

```
│   │   confusion_matrix.png        Ma trận nhầm lẫn — bảng đối chiếu bệnh máy
│   │                              đoán so với bệnh thật, giúp thấy máy hay
│   │                              nhầm lớp nào với lớp nào
│   └── weights/best.pt, best.onnx   Trọng số (weights — các con số máy học được
│                                  sau khi huấn luyện, quyết định cách nó dự
│                                  đoán) của model tốt nhất, lưu 2 định dạng:
│                                  PyTorch (định dạng gốc, dùng để huấn luyện
│                                  tiếp) và ONNX (Open Neural Network Exchange
│                                  — định dạng đã chuyển đổi, gọn nhẹ hơn, dùng
│                                  để chạy dự đoán thực tế)
│
├── result/                      Kết quả chạy thử dự đoán (inference — quá trình
│                                đưa ảnh vào model đã huấn luyện để lấy kết quả,
│                                khác với huấn luyện là không học thêm gì nữa)
│                                trên ảnh mẫu, nhiều lần chạy, mỗi thư mục con là
│                                1 lần thử, kèm bảng tổng hợp và biểu đồ độ chính xác
│
├── datasets_samples/            Bản sao làm việc của backend (giống hệt deploy_pi/)
├── deploy_pi/                   ★ Bản đóng gói CHÍNH THỨC để copy sang Raspberry Pi
│   ├── server/                      Backend (phần xử lý phía máy chủ, chạy ngầm,
│   │                                không phải giao diện người dùng trực tiếp
│   │                                nhìn thấy), viết bằng Flask (thư viện Python
│   │                                dựng web server nhỏ gọn)
│   │   ├── app.py                       Điểm khởi động chương trình: định nghĩa
│   │   │                                các địa chỉ (route) mà dashboard gọi tới,
│   │   │                                khởi động luồng chụp ảnh + luồng dự đoán
│   │   ├── config.py                    Toàn bộ cấu hình (đường dẫn file, chọn
│   │   │                                camera, ngưỡng độ tin cậy, tên lớp bệnh)
│   │   ├── camera.py                    Xử lý các loại camera khác nhau (webcam
│   │   │                                USB / camera rời gắn bo mạch Pi) theo
│   │   │                                một giao diện thống nhất
│   │   ├── inference.py                 Nạp model đã huấn luyện (định dạng ONNX),
│   │   │                                xử lý ảnh đầu vào, chạy dự đoán
│   │   ├── sensors.py                   Đọc dữ liệu cảm biến đất/khí hậu (hiện
│   │   │                                đang tạo số giả lập, chờ cắm cảm biến
│   │   │                                phần cứng thật — xem phần 4)
│   │   ├── chatbot.py                   Trợ lý tư vấn trả lời theo luật cố định
│   │   │                                (rule-based — trả lời theo các quy tắc
│   │   │                                lập trình sẵn, không dùng AI tạo sinh)
│   │   └── requirements.txt             Danh sách thư viện Python cần cài
│   ├── templates/agrismart/         Giao diện dashboard tĩnh, chỉ dùng HTML/CSS/
│   │                                JavaScript cơ bản, không dùng framework (bộ
│   │                                khung dựng sẵn như React, Vue...)
│   ├── paddy_classify/weights/best.onnx   Model đã huấn luyện, đã chuyển sang
│   │                                định dạng ONNX (~21 MB)
│   └── sample_images/               120 ảnh mẫu (12 ảnh × 10 loại bệnh) để thử
│                                    nghiệm khi chưa lắp camera thật
│
├── templates/agrismart-dashboard.html   Bản dashboard gộp trong 1 file (bản cũ,
│                                 không còn dùng)
└── DEPLOY.md                     Hướng dẫn nhanh: cần copy gì sang Pi, triển khai
                                 (deploy — đưa chương trình lên môi trường chạy
                                 thật) như thế nào
```

> `datasets_samples/` và `deploy_pi/` hiện có nội dung **giống hệt nhau** (đã so sánh từng byte — đơn vị dữ liệu nhỏ nhất trong máy tính — và không có khác biệt) — `deploy_pi/` là bản chính thức dùng để copy sang Pi, `datasets_samples/` là bản làm việc song song trên máy phát triển (dev). Sửa code thì nên sửa cả hai, hoặc hợp nhất lại để tránh lệch nhau.

1.2 Cơ chế hoạt động chung (backend)

File khởi động chính (`app.py`) chạy 2 luồng xử lý nền (thread — một tiến trình con chạy song song với chương trình chính) cùng lúc, dùng chung một vùng lưu kết quả và một cơ chế khoá (đảm bảo tại một thời điểm chỉ một luồng được ghi vào vùng dữ liệu chung, tránh 2 luồng ghi đè lẫn nhau):

- **Luồng chụp ảnh** (xử lý trong `camera.py`) — đọc khung hình (frame — một ảnh đơn tại một thời điểm, video là một chuỗi khung hình nối tiếp nhau) liên tục từ camera (webcam USB hoặc camera CSI — Camera Serial Interface, cổng cắm camera rời tích hợp sẵn trên bo mạch Raspberry Pi), luôn giữ khung hình mới nhất trong bộ nhớ. Việc đọc liên tục này phục vụ riêng cho luồng video gửi lên dashboard, tách biệt khỏi việc chạy model để không tranh chấp thiết bị camera.
- **Luồng dự đoán** — cứ mỗi khoảng thời gian cấu hình sẵn (mặc định 1,5 giây) lại lấy khung hình mới nhất, đưa qua model đã huấn luyện (chạy bằng ONNX Runtime — công cụ chạy model ở định dạng ONNX, chỉ cần bộ xử lý trung tâm — CPU, Central Processing Unit — không cần card đồ hoạ), rồi cập nhật kết quả (tên bệnh, độ tin cậy, mức độ nghiêm trọng) vào vùng dữ liệu dùng chung.

Server cung cấp 4 địa chỉ chính (API endpoint — địa chỉ để chương trình khác, ở đây là giao diện dashboard, gọi vào lấy hoặc gửi dữ liệu). Dashboard (viết bằng JavaScript thuần trong `templates/agrismart/js/`) tự động gọi lại các địa chỉ này theo chu kỳ đều đặn (cơ chế gọi là "polling" — hỏi lại định kỳ, khác với việc server chủ động đẩy dữ liệu xuống) để cập nhật giao diện:

Địa chỉ (route)	Chức năng
`GET /video_feed`	Luồng video trực tiếp từ camera, dạng MJPEG (Motion JPEG — gửi liên tục nhiều ảnh JPEG rời để tạo cảm giác video, không phải file video thật)
`GE Tapi/predict/latest`	Kết quả phân loại bệnh mới nhất (đọc từ dữ liệu đã tính sẵn, không chạy lại model mỗi lần gọi)
`GET /api/sensors`	Dữ liệu đất/khí hậu: hàm lượng N-P-K (ba dưỡng chất chính trong đất: Đạm — N, Lân — P, Kali — K), độ pH (thang đo độ chua/kiềm của đất), nhiệt độ, độ ẩm — hiện là số giả lập dao động quanh mức thực tế ngoài đồng
	Chatbot trả lời theo luật cố định, dựa trên bệnh đang phát hiện và

`POST /api/chat`	Chatbot trả lời theo luật cố định, dựa trên bệnh đang phát hiện và từ khoá trong câu hỏi

Thiết kế này tách rời hoàn toàn việc "truyền video" và "chạy model", nên dashboard vẫn hiển thị mượt (25 khung hình/giây — fps, frames per second, đơn vị đo độ mượt video) kể cả khi model học sâu chạy chậm trên CPU của Pi.

---

2. Cài đặt & chạy end-to-end (từ đầu đến cuối)

2.1 Chạy thử nhanh trên máy phát triển (không cần camera thật)

```
cd deploy_pi          # hoặc datasets_samples – nội dung giống nhau
python -m venv .venv  # tạo môi trường ảo (venv – virtual environment, nơi cài
                       # riêng thư viện Python cho dự án này, không ảnh hưởng
                       # các dự án khác trên cùng máy)
# Windows: .venv\Scripts\activate | Linux/Mac: source .venv/bin/activate
pip install -r server/requirements.txt

# Thử model trên ảnh mẫu có sẵn (không cần camera)
python server/inference.py sample_images/blast/*.jpg
# hoặc không truyền tham số -> tự lấy 1 ảnh mỗi loại trong paddy_images/
```

Chạy cả server (dashboard sẽ hiện ảnh chờ vì chưa có camera):

```
python server/app.py
# mở http://localhost:5000
```

2.2 Triển khai thật lên Raspberry Pi 4

```
# 1. Copy nguyên thư mục deploy_pi/ sang Pi (qua scp hoặc USB), ví dụ để ở ~/agrismart
cd ~/agrismart

# 2. Cài hệ thống + môi trường ảo Python
sudo apt update && sudo apt install -y python3-venv python3-pip
sudo apt install -y python3-picamera2      # chỉ cần nếu dùng camera rời gắn bo mạch Pi
python3 -m venv .venv --system-site-packages   # cờ này giúp venv thấy được thư viện picam
source .venv/bin/activate
pip install -r server/requirements.txt

# 3. Chọn nguồn camera (mặc định "auto": thử camera gắn bo mạch Pi trước, không có thì dùr
export CAMERA_SOURCE=usb        # hoặc picam (camera gắn bo mạch Pi)
export USB_CAMERA_INDEX=0       # số thứ tự webcam nếu máy có nhiều camera USB

# 4. Chạy
python server/app.py            # mở http://<địa-chỉ-IP-của-Pi>:5000
```

Chi tiết đầy đủ (chọn camera, xem từ xa qua mạng 4G, tự khởi động cùng hệ thống) xem deploy_pi/server/README.md và DEPLOY.md.

2.3 Huấn luyện lại model từ đầu (nếu cần)

Repo (kho mã nguồn) này **không chứa mã lệnh huấn luyện** — quá trình huấn luyện chạy trên một máy khác có card đồ hoạ (GPU — Graphics Processing Unit, phần cứng chuyên tính toán song song, giúp huấn luyện model nhanh hơn CPU rất nhiều), thuê trên nền tảng đám mây (đường dẫn ghi trong `result_training/args.yaml` là `/workspace/benholua/...` cho thấy điều này). Máy hiện tại chỉ lưu lại dữ liệu ảnh gốc và kết quả đã huấn luyện xong. Để huấn luyện lại:

```
pip install ultralytics   # thư viện chứa sẵn kiến trúc YOLO và công cụ huấn luyện

# Cấu trúc thư mục ảnh theo chuẩn phân loại ảnh của thư viện Ultralytics:
#   dataset_yolo/train/<tên_lớp_bệnh>/*.jpg
#   dataset_yolo/val/<tên_lớp_bệnh>/*.jpg
#   (chia từ paddy_images/ theo tỉ lệ train/val, xem so_do_mo_hinh.md sơ đồ 2: 70/15/15)

yolo classify train \
  model=yolo11s-cls.pt \
  data=dataset_yolo \
  epochs=100 imgsz=224 batch=32 patience=10 optimizer=AdamW \
  project=runs name=paddy_classify

# Xuất (export) sang định dạng ONNX để chạy nhẹ trên Pi — chỉ cần thư viện
# onnxruntime, không cần cài PyTorch/Ultralytics trên thiết bị chạy thật
yolo export model=runs/paddy_classify/weights/best.pt format=onnx simplify=true
```

Giải thích nhanh các tham số huấn luyện chính ở trên:

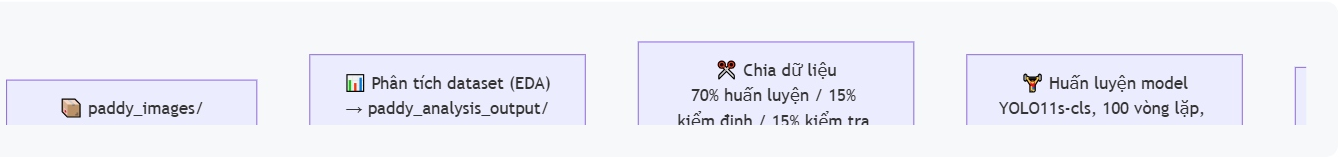
- `epochs=100`: số vòng lặp huấn luyện (epoch), mỗi vòng máy nhìn qua hết toàn bộ ảnh huấn luyện một lần.
- `imgsz=224`: kích thước ảnh đầu vào — mọi ảnh được co giãn về 224×224 pixel trước khi đưa vào model.
- `batch=32`: số ảnh xử lý cùng lúc trong mỗi bước huấn luyện.
- `patience=10`: dừng sớm nếu sau 10 vòng liên tiếp độ chính xác không cải thiện, tránh huấn luyện dư thừa.
- `optimizer=AdamW`: thuật toán tối ưu (optimizer — quy tắc toán học điều chỉnh trọng số model sau mỗi bước huấn luyện) tên AdamW, một biến thể phổ biến của thuật toán Adam.

Toàn bộ tham số đã dùng cho lần huấn luyện hiện tại (tốc độ học, các kỹ thuật tăng cường dữ liệu, seed — số khởi tạo ngẫu nhiên giúp kết quả lặp lại được...) xem result_training/args.yaml. Sau khi huấn luyện xong, copy file `best.onnx` mới vào

`deploy_pi/paddy_classify/weights/best.onnx`.

---

3. Luồng thiết kế dự án



Ý nghĩa từng bước, đối chiếu với file thực tế trong repo:

1. **Chuẩn bị dữ liệu** — `paddy_images/` (dataset gốc) → phân tích thống kê tại `paddy_analysis_output/` để hiểu mức mất cân bằng lớp (tỉ lệ 5,23 lần giữa lớp có nhiều ảnh nhất và lớp ít ảnh nhất), phân bố giống lúa/tuổi cây trước khi huấn luyện.
2. **Chia tập huấn luyện / kiểm định / kiểm tra cuối** — theo sơ đồ trong so_do_mo_hinh.md (tỉ lệ 70/15/15, giữ đều tỉ lệ các lớp bệnh ở mỗi phần). Bước này chạy ở máy huấn luyện, không có sẵn trong repo hiện tại.
3. **Huấn luyện** — dùng thư viện Ultralytics, kiến trúc YOLO11s-cls, 100 vòng lặp (dừng sớm nếu 10 vòng liên tiếp không cải thiện), ảnh 224×224 pixel, kèm các kỹ thuật tăng cường dữ liệu (augmentation — biến đổi ngẫu nhiên ảnh gốc để tạo thêm tình huống cho máy học, ví dụ: ghép nhiều ảnh lại — mosaic, lật ảnh — flip, đổi màu sắc — HSV (Hue-Saturation-Value: Sắc độ — Độ bão hoà — Độ sáng), xoá ngẫu nhiên một vùng ảnh — random-erasing). Tham số đầy đủ: result_training/args.yaml.
4. **Đánh giá** — theo dõi độ chính xác top-1 (tỉ lệ máy đoán đúng ngay ở lựa chọn có xác suất cao nhất) và top-5 (tỉ lệ đáp án đúng nằm trong 5 lựa chọn có xác suất cao nhất máy đưa ra) sau mỗi vòng huấn luyện (results.csv, đạt khoảng 98% top-1 / 100% top-5 ở vòng cuối), soi ma trận nhầm lẫn để phát hiện lớp bệnh hay bị đoán lẫn với nhau.
5. **Xuất định dạng (export)** — `best.pt` (định dạng PyTorch, dùng để huấn luyện tiếp/đánh giá) → `best.onnx` (định dạng ONNX, dùng để chạy dự đoán, không cần cài PyTorch/Ultralytics trên thiết bị chạy thật).
6. **Tích hợp vào backend** — `inference.py` nạp model ONNX bằng thư viện onnxruntime, xử lý ảnh đầu vào giống hệt lúc huấn luyện: co giãn theo cạnh ngắn → cắt phần chính giữa ảnh theo kích thước cố định (center-crop) → chuẩn hoá giá trị điểm ảnh (normalize — đưa về khoảng số chuẩn để model dễ xử lý).
7. **Đóng gói triển khai** — `deploy_pi/` gộp server + dashboard + model + ảnh mẫu thành 1 thư mục tự chứa (~36 MB), không phụ thuộc phần còn lại của repo.
8. **Vận hành thực tế** — chạy trên Raspberry Pi ngoài đồng: camera chụp liên tục → model dự đoán định kỳ → dashboard hiển thị kết quả + chatbot tư vấn xử lý.

---

4. Triển khai phần cứng

Hệ thống được thiết kế chạy **hoàn toàn cục bộ trên Raspberry Pi 4** (không cần card đồ hoạ, không cần gửi ảnh lên máy chủ đám mây để dự đoán) — phù hợp lắp đặt ngay ngoài đồng ruộng.

Thành phần	Vai trò	Trạng thái hiện tại
Raspberry Pi 4 (máy tính nhúng cỡ nhỏ, giá rẻ, đủ mạnh để chạy Linux đầy đủ)	Chạy backend Flask + model ONNX bằng CPU	Mục tiêu triển khai chính
Camera	Chụp ảnh lá lúa liên tục	2 lựa chọn, tự động dò qua cấu hình `CAMERA_SOURCE=auto`:
— Webcam USB Hikvision	Kết nối qua cổng USB, đọc ảnh bằng thư viện OpenCV (thư viện xử lý ảnh/video phổ biến)	Đã hỗ trợ sẵn trong `camera.py`
— Camera CSI (Camera Serial Interface — cổng cắm camera rời tích hợp sẵn trên bo mạch Pi)	Đọc ảnh qua thư viện picamera2 dành riêng cho phần cứng Pi	Đã hỗ trợ sẵn trong `camera.py`, cần cài qua công cụ quản lý gói hệ điều hành (`apt`), không cài qua `pip` (công cụ cài thư viện Python)
Cảm biến đất/khí hậu	Đo N-P-K (Đạm — Lân — Kali, ba dưỡng chất chính trong đất), độ pH, nhiệt độ, độ ẩm	**Đang là số giả lập** (`sensors.py` tạo số ngẫu nhiên dao động quanh mức thực tế ngoài đồng). Khi có phần cứng cảm biến NPK thật (kết nối theo chuẩn RS485/Modbus — chuẩn giao tiếp có dây phổ biến trong công nghiệp) và cảm biến nhiệt-ẩm thật (ví dụ DHT22/SHT3x, kết nối theo chuẩn I2C/UART — các chuẩn giao tiếp phổ biến giữa vi điều khiển và cảm biến), chỉ cần thay phần đọc dữ liệu bên trong `sensors.py` — giao diện gọi ra bên ngoài và dashboard giữ nguyên
	Xem dashboard qua điện thoại khi đang ở ngoài đồng, dùng	

Kết nối mạng từ xa	ngoài đồng, dùng SIM 4G (module SIM7600X) — do nhà mạng dùng cơ chế CGNAT (Carrier-Grade NAT: nhiều thuê bao dùng chung 1 địa chỉ IP công khai) nên không thể mở cổng mạng (port) để truy cập trực tiếp từ ngoài vào	Dùng **Cloudflare Tunnel** (dịch vụ miễn phí tạo một "đường hầm" kết nối máy trong mạng nội bộ ra Internet mà không cần mở cổng) hoặc **Tailscale** (dịch vụ tạo mạng riêng ảo — VPN — kết nối bảo mật giữa các thiết bị của người dùng)
Tự khởi động	Backend tự chạy lại khi Pi mất điện hoặc khởi động lại	Cấu hình qua systemd (hệ thống quản lý tiến trình nền có sẵn trên Linux, dùng để tự chạy chương trình khi máy khởi động) — file dịch vụ `agrismart.service`, xem `deploy_pi/server/README.md`

Chi tiết cấu hình từng phần (chọn camera, cài công cụ `cloudflared` cho kiến trúc chip ARM64 của Pi, nội dung file dịch vụ systemd) xem đầy đủ tại deploy_pi/server/README.md mục 2, 4, 5.